

PROJECT REPORT

ExamProctor: A Full-Stack Web-Based Automated Examination and Proctoring System

Submitted by

Mohammad Asif

Registration Number: 12515921

Course Code: PETV137

Under the Guidance of

Priyanka Gupta, Dr Harpeet Kaur

School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab

DECLARATION

I hereby declare that the project work entitled “ExamProctor: A Full-Stack Web-Based Automated Examination and Proctoring System” is an authentic record of my own work carried out as a requirement of the Project for the award of the MCA Degree in Course Code PETV137 from Lovely Professional University, Phagwara, under the guidance of Priyanka Gupta, Dr Harpeet Kaur, during June to July 2026. All the information furnished in this project report is based on my own intensive work and is genuine.

Name of Student: Mohammad Asif

Registration Number: 12515921

Date:
12 July 2026

CERTIFICATE

This is to certify that the declaration statement made by the student is correct to the best of my knowledge and belief. He has completed this Project under my guidance and supervision. The present work is the result of his original investigation and effort. No part of this work has been submitted for any other degree at any University. The Project is fit for submission and partial fulfilment of the conditions for the award of the MCA Degree from Lovely Professional University, Phagwara.

Signature and Name of the Mentor

Designation

School of Computer Science and Engineering,
Lovely Professional University, Phagwara, Punjab.

Date: 12 July 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty mentor for the continuous support, patience, and valuable guidance extended throughout the duration of this project. Their insights into system design and their willingness to review my work at every stage played a significant role in shaping the final outcome.

I am also thankful to the School of Computer Science and Engineering, Lovely Professional University, for providing the infrastructure and academic environment necessary to carry out this work. Finally, I extend my appreciation to my peers and family for their encouragement during the course of this project.

Mohammad Asif

ABSTRACT

Online examinations have become an integral part of academic and corporate assessment, yet most existing platforms either depend heavily on manual proctoring or lack an integrated environment for exam creation, candidate management, and result publication. This project, ExamProctor, presents a full-stack web-based examination portal that automates test integrity monitoring through browser-level tab-focus detection while consolidating student, examiner, and administrative workflows into a single role-based application.

The system is built using React 19 with Vite on the client side, and a Node.js/Express backend exposing REST endpoints secured through JSON Web Tokens. A dual-database architecture separates transactional application data, stored in SQLite, from audit and proctoring-event logs, stored in MongoDB, while Socket.io provides real-time delivery of tab-switch warnings and proctoring alerts to examiners. The entire application is containerised using Docker and orchestrated through Docker Compose, with an automated Jenkins CI/CD pipeline triggered via GitHub webhooks handling build, image publishing, and deployment.

The resulting platform demonstrates that automated browser-focus proctoring, combined with a consolidated role-based portal and a fully automated deployment pipeline, can meaningfully reduce the manual overhead associated with conducting secure online assessments, while remaining lightweight enough to run on a single host during development and testing.

TABLE OF CONTENTS

1. INTRODUCTION	7
1.1 Background of the Study	7
1.2 Objectives of the Project	8
2. SCOPE OF THE STUDY (PROBLEM STATEMENT)	9
3. EXISTING SYSTEM	10
3.1 Introduction	10
3.2 Existing Software	10
3.3 DFD for Present System	10
3.4 What's New in the System to Be Developed	11
4. PROBLEM ANALYSIS	12
4.1 Product Definition	12
4.2 Feasibility Analysis	12
5. SOFTWARE REQUIREMENT ANALYSIS	13
5.1 Introduction	13
5.2 General Description	13
5.3 Specific Requirements	13
6. DESIGN	15
6.1 System Design	15
6.2 Design Notations	15
6.3 Detailed Design	16
6.4 Flowcharts	17
6.5 Pseudo Code	18
7. TESTING	20
7.1 Functional Testing	20
7.2 Structural Testing	20
7.3 Levels of Testing	20
7.4 Testing the Project	20
8. IMPLEMENTATION	22
8.1 Implementation of the Project	22
8.2 Conversion Plan	23
8.3 Post-implementation and Software Maintenance	24
9. PROJECT LEGACY	25
9.1 Current Status of the Project	25
9.2 Remaining Areas of Concern	25
9.3 Technical and Managerial Lessons Learnt	25
10. BIBLIOGRAPHY	26

1. INTRODUCTION

Assessment has always been at the heart of academic and professional certification, and the manner in which it is conducted has evolved considerably over the last decade. What was once confined to physical examination halls with invigilators walking between rows of desks has, particularly since 2020, shifted substantially toward online delivery. This shift brought with it a new class of problem: how does an institution verify that a candidate sitting an exam from a personal device, in an uncontrolled environment, is not consulting unauthorised material or another person during the assessment window?

ExamProctor was conceived to address this problem directly. It is a full-stack web application that combines conventional examination-portal functionality — exam creation, candidate scheduling, submission handling, and grading — with an automated proctoring layer built around browser tab-focus detection. Rather than requiring a human invigilator to watch a video feed for the entire duration of a test, the system observes browser-level signals such as tab switches and window-blur events, flags them in real time, and automatically submits the exam once a configurable violation threshold is crossed.

1.1 Background of the Study

The motivation for this project stems from direct observation of how existing institutional examination tools are typically structured. Many are legacy systems retrofitted with a video-proctoring add-on, resulting in disjointed workflows where student records, exam configuration, and violation logs live in separate, loosely integrated modules. Smaller institutions and independent examiners, meanwhile, often have no proctoring capability at all beyond trusting the candidate. Between these two extremes lies an opportunity for a lightweight, self-hosted system that an examiner can deploy without depending on a third-party SaaS subscription, while still offering meaningful integrity safeguards.

A second consideration that shaped this project was deployment complexity. Full-stack applications with a frontend, backend, and database are frequently difficult to set up consistently across a developer's laptop, a staging server, and a production host. Configuration drift — the classic “it works on my machine” problem — was treated as a first-class concern from the outset, which is why the project incorporates Docker-based containerisation and an automated Jenkins pipeline as core components rather than an afterthought.

1.2 Objectives of the Project

The project was guided by one overarching goal, broken down into a small set of concrete, measurable objectives.

1.2.1 primary objective

To design and implement a secure, role-based web examination platform that automates integrity monitoring through tab-focus proctoring, while remaining simple enough to deploy on a single host or scale across containers as demand grows.

1.2.2 specific objectives

- To implement distinct, permission-scoped portals for Student, Examiner, and Admin/Super Admin roles.
- To build an automated tab-switch and window-blur detection mechanism that increments a warning counter and auto-submits an exam once a configurable threshold is exceeded.
- To provide examiners with tools to create exams, manage associated students, review proctoring violation logs, and publish grades.
- To secure all API communication using JSON Web Tokens and to hash stored credentials using bcrypt.
- To containerise the frontend and backend services using Docker and to orchestrate them with Docker Compose.
- To automate the build, test, and deployment lifecycle using a declarative Jenkins pipeline triggered by GitHub webhooks.

2. SCOPE OF THE STUDY (PROBLEM STATEMENT)

Educational institutions and independent examiners conducting online assessments face a recurring set of operational difficulties. Candidates can trivially open a second browser tab to search for answers, communicate with a third party, or consult reference material, and without continuous supervision this behaviour frequently goes undetected. Assigning a human proctor to every candidate is expensive and does not scale beyond a handful of simultaneous test-takers. At the same time, the systems used to create exams, manage student records, and publish grades are often maintained as separate tools, forcing examiners to reconcile data manually across spreadsheets, learning-management systems, and grading portals.

The problem this project addresses can therefore be stated as follows: existing online examination workflows lack an integrated, low-overhead mechanism for detecting integrity violations in real time, and the tooling required to create, administer, and grade exams is frequently fragmented across multiple disconnected systems.

The scope of ExamProctor is deliberately bounded to keep the system achievable within an academic project timeline while still addressing the core of this problem. It covers browser-based tab-focus and window-blur proctoring (not full video-based facial recognition, which was identified as a possible future enhancement rather than a core deliverable), consolidated role-based portals for students, examiners, and administrators, JWT-secured REST APIs for all data operations, and a fully containerised deployment pipeline. Video proctoring, plagiarism detection in free-text answers, and mobile-native applications were considered but placed outside the boundary of the current implementation, both to manage scope and because they would require substantially different infrastructure — object storage for video, for instance — than what a self-hosted academic project can reasonably provision.

3. EXISTING SYSTEM

3.1 Introduction

Before finalising the design of ExamProctor, it was necessary to examine how examinations are currently conducted in the absence of a dedicated platform, and to review the class of proctoring tools already available in the market. This chapter summarises both the manual process that many smaller institutions still rely on and the commercial software landscape, before identifying what the proposed system contributes that these alternatives do not.

3.2 Existing Software

Commercial proctoring platforms such as ProctorU, Examity, and Honorlock dominate the enterprise segment. These tools typically rely on continuous webcam and microphone monitoring, AI-based gaze tracking, and in some cases live human proctors watching multiple candidate feeds simultaneously. While effective, they are costly to license, require significant bandwidth for continuous video streaming, and are generally offered as closed, hosted SaaS products that institutions cannot self-host or customise. On the other end of the spectrum, many college-level and internal corporate assessments are still administered through generic form tools or learning-management-system quiz modules that offer no proctoring capability whatsoever, relying instead on an honour system or a live video call supervised by a human invigilator.

3.3 DFD for Present System

The diagram below models the manual examination process typical of an institution without a dedicated proctoring platform: a coordinator schedules the exam, students receive the paper or link over email, submissions are collected manually, and grading is performed offline before results are communicated back to students.

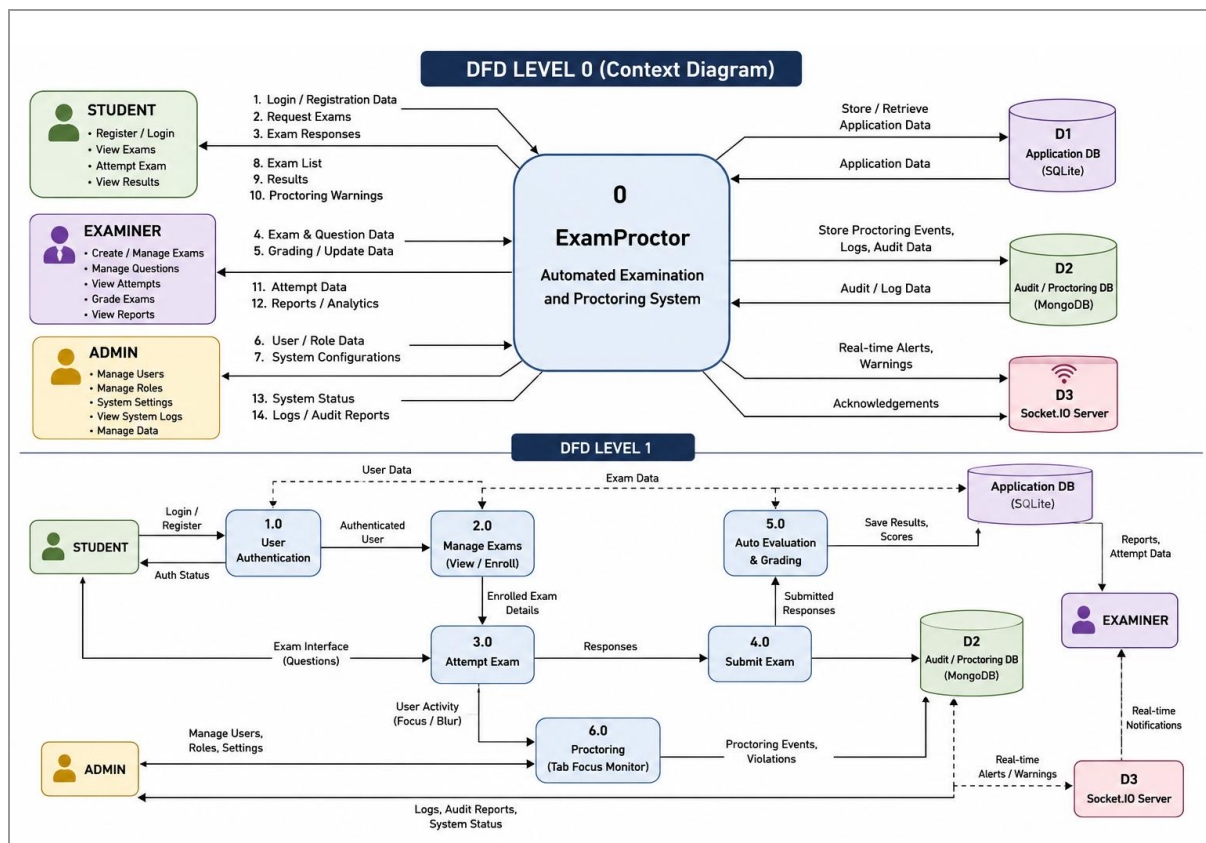


Figure 3.1: Data Flow Diagram of the Present (Manual) Examination Process

3.4 What's New in the System to Be Developed

ExamProctor departs from both extremes described above. Unlike commercial video-proctoring suites, it uses a lightweight, browser-native signal — tab-focus and window-blur events — that requires no continuous video upload, no dedicated bandwidth allocation for streaming, and no third-party licensing cost. Unlike the manual, honour-system-based approach common in smaller institutions, it introduces automated, real-time violation detection with a configurable sensitivity threshold that administrators control centrally. Additionally, by consolidating exam creation, student-examiner association, grading, and audit logging into a single role-based portal, the system removes the need to reconcile data across separate spreadsheets or disconnected tools, and by shipping with a Docker Compose configuration and a Jenkins pipeline, it can be deployed and kept up to date with substantially less operational effort than a bespoke, manually maintained deployment.

4. PROBLEM ANALYSIS

4.1 Product Definition

ExamProctor is defined as a self-hosted, full-stack web application providing three role-scoped portals — Student, Examiner, and Admin/Super Admin — built around a proctored examination workflow. The product's defining characteristic is its automated tab-focus proctoring mechanism, which observes browser visibility and focus events during an active exam session, increments a warning counter on each violation, and triggers an automatic submission once the examiner- or administrator-defined threshold is exceeded. The product also serves as a system of record for exam configuration, student-examiner association, submission data, grades, and proctoring audit logs.

4.2 Feasibility Analysis

A feasibility assessment was carried out across the standard technical, economic, and operational dimensions before development began, summarised in Table 4.1.

Dimension	Assessment
Technical	All required technologies — React, Node.js/Express, SQLite, MongoDB, Docker, Jenkins — are mature, well-documented, and freely available. No proprietary or licensed component is required, and the development team already had working familiarity with the JavaScript ecosystem across the stack.
Economic	The system uses entirely open-source software and can be self-hosted on commodity hardware or a low-cost cloud instance, avoiding the recurring per-seat licensing fees charged by commercial proctoring vendors.
Operational	Docker Compose reduces environment setup to a single command, and the Jenkins pipeline automates build and deployment, minimising the ongoing operational burden on whoever administers the platform after handover.
Schedule	The scope was deliberately bounded (Chapter 2) to fit within a single academic project term, deferring video-based proctoring and mobile applications to future work.

5. SOFTWARE REQUIREMENT ANALYSIS

5.1 Introduction

This chapter documents the functional and non-functional requirements that governed the design and implementation of ExamProctor. Requirements were derived from the problem statement in Chapter 2 and refined against the three defined user roles: Student, Examiner, and Admin/Super Admin.

5.2 General Description

ExamProctor operates as a client-server web application. The client is a single-page React application served through Nginx, and the server is a Node.js/Express REST API that additionally exposes a Socket.io channel for real-time event delivery. The product functions within a browser environment and assumes each candidate has access to a desktop or laptop web browser with camera access permitted where webcam snapshotting is enabled by the administrator. Table 5.1 summarises the minimum hardware and software environment assumed for both development and deployment.

Component	Requirement
Development Machine	4 GB RAM minimum (8 GB recommended), dual-core processor, 10 GB free disk space
Server / Deployment Host	2 vCPU, 4 GB RAM minimum for Docker Compose stack (frontend, backend, MongoDB)
Operating System	Any OS supporting Docker Engine (Linux, macOS, Windows with WSL2)
Runtime	Node.js 20.x, npm 10.x
Browser (Client)	A modern evergreen browser supporting the Page Visibility API and WebSocket connections (Chrome, Firefox, Edge)
Database	SQLite3 (bundled, file-based) and MongoDB 6.x (audit store)

5.3 Specific Requirements

Functional requirements describe the observable behaviour the system must exhibit for each role, and are summarised in Table 5.2. Non-functional requirements address the qualities the system must possess independent of any specific feature: the API must respond to standard CRUD operations within two seconds under normal load; JWT tokens must expire after twenty-four hours to bound the exposure window of a compromised token;

passwords must never be stored or logged in plain text; and the containerised services must be recoverable through a documented Docker Compose command sequence without manual reconfiguration.

ID	Requirement	Applicable Role(s)
FR-1	Register and authenticate using a JWT-secured login endpoint	Student, Examiner, Admin
FR-2	Select an assigned examiner during registration	Student
FR-3	Create, configure, and publish exams with timing and subject metadata	Examiner
FR-4	Attempt an exam within a proctored session and receive real-time tab-switch warnings	Student
FR-5	Auto-submit the exam upon exceeding the configured tab-switch threshold	System
FR-6	Review proctoring violation logs with timestamps per submission	Examiner
FR-7	Grade submissions and publish results to the student dashboard	Examiner
FR-8	Configure system-wide settings: registration toggle, proctoring sensitivity, max tab switches	Admin
FR-9	View platform-wide usage metrics and perform database maintenance	Admin / Super Admin

6. DESIGN

6.1 System Design

ExamProctor follows a three-tier architecture composed of a presentation tier (the React single-page application), an application tier (the Node.js/Express REST API together with a Socket.io real-time channel), and a data tier split across two purpose-specific stores: SQLite for transactional application data — users, exams, and submissions — and MongoDB for schema-flexible audit and proctoring-event logging. This separation was a deliberate design decision: transactional records benefit from SQLite's simplicity and file-based portability during development, while audit trails benefit from MongoDB's document flexibility when logging heterogeneous proctoring events that may carry different metadata depending on the violation type.

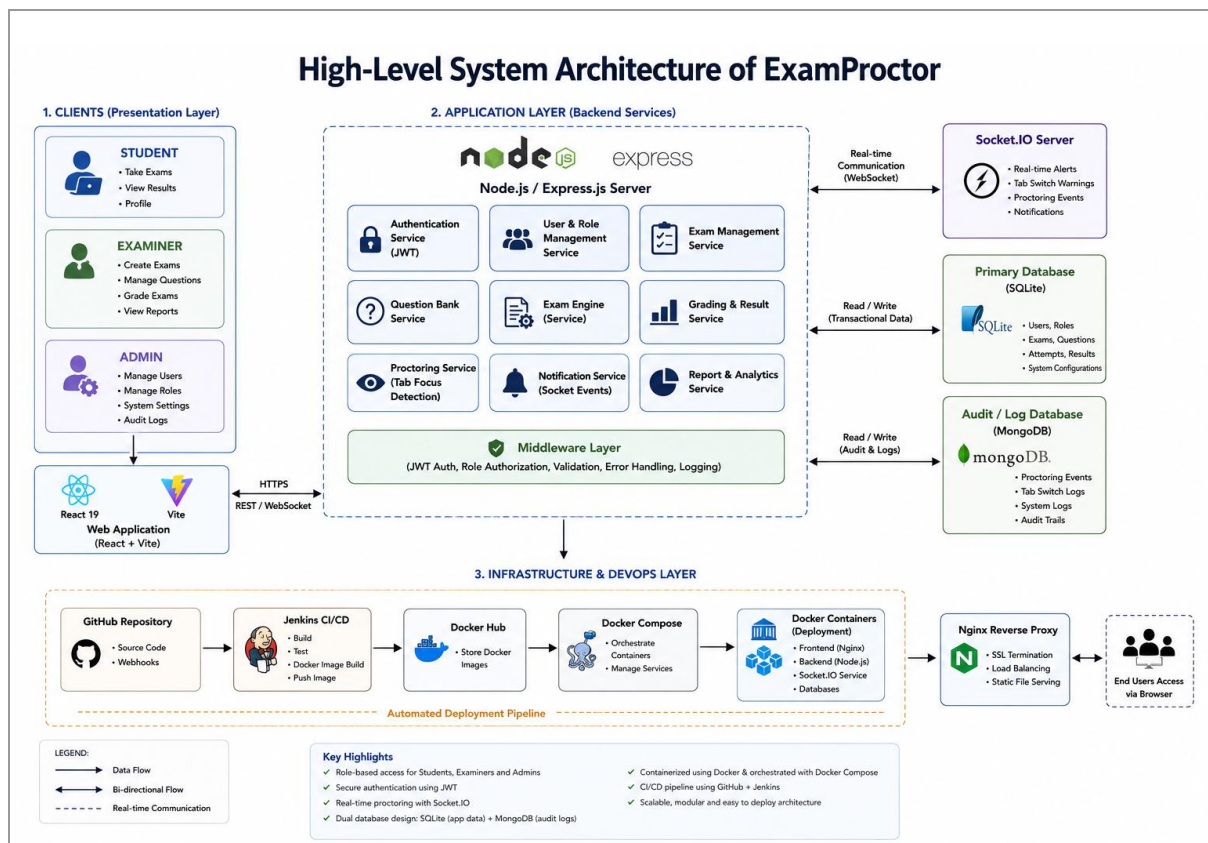


Figure 6.1: High-Level System Architecture of ExamProctor

6.2 Design Notations

The design of the system is expressed using three standard notations: Entity-Relationship (ER) diagrams to model the persistent data schema, Unified Modelling Language (UML) sequence diagrams to describe the interaction between the client, server,

and database across time, and flowcharts to describe the decision logic embedded in the proctoring mechanism. These notations were chosen because they are widely understood, map cleanly onto the REST and event-driven interactions used throughout the system, and are well supported by the diagramming tools used during development.

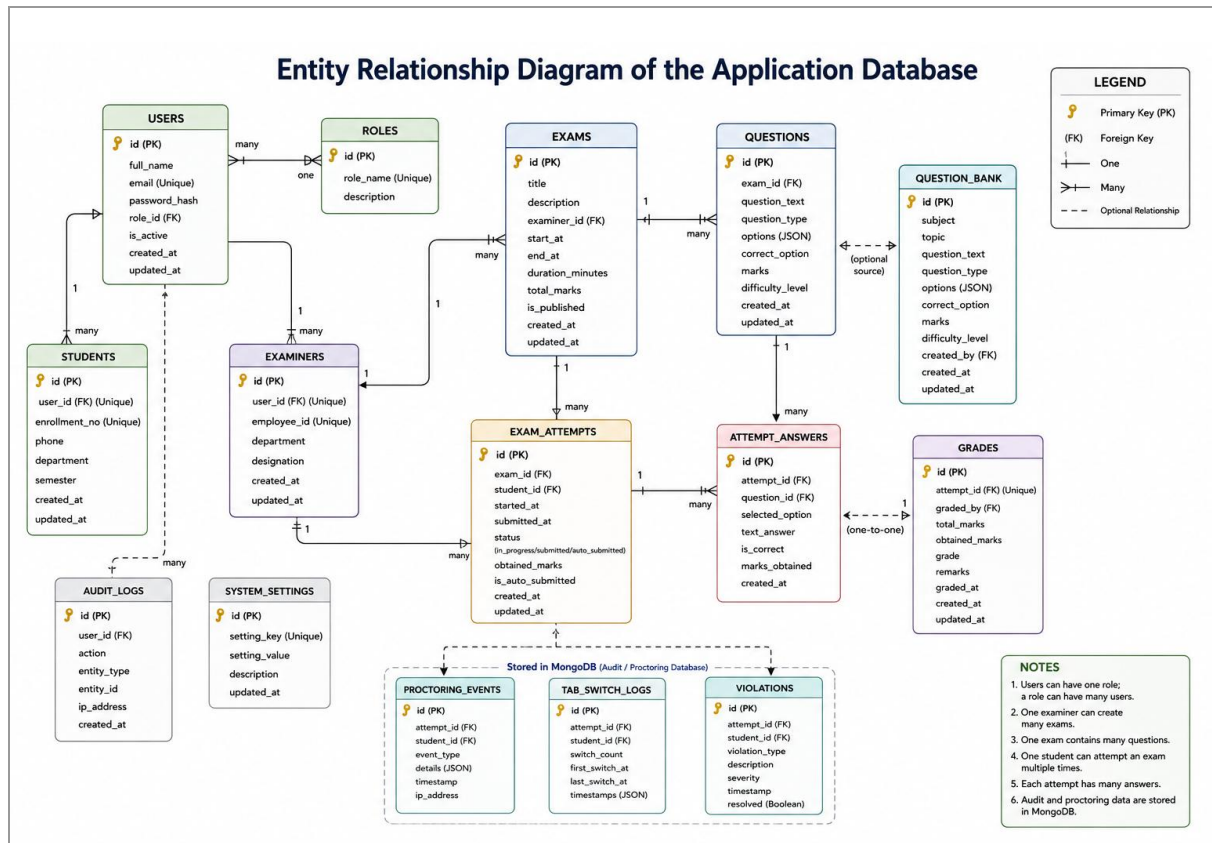


Figure 6.2: Entity Relationship Diagram of the Application Database

6.3 Detailed Design

The authentication subsystem issues a signed JWT on successful login, embedding the user's identifier and role as claims. Every subsequent request to a protected route passes this token in the Authorization header, where a custom Express middleware verifies the signature, checks expiry, and attaches the decoded payload to the request object before handing control to the relevant controller. Role-based access control is enforced at two layers: client-side route guards prevent unauthorised navigation, and server-side middleware independently re-validates the role on every request, so that a compromised or tampered client cannot bypass authorisation simply by manipulating the frontend.

The exam submission subsystem accepts a payload containing the candidate's answers together with the accumulated proctoring event log for that session. The controller validates the payload structure, persists the submission and its associated grade placeholder to SQLite,

and asynchronously writes the detailed proctoring event trail to MongoDB for later review by the examiner. A Socket.io event is emitted in parallel so that, if the examiner is actively monitoring a live session, tab-switch warnings appear on their dashboard without requiring a page refresh.

6.4 Flowcharts

The core integrity-monitoring logic of the application is the tab-focus proctoring flow, illustrated below. The client binds listeners to the browser's `visibilitychange` and `blur` events; each time either fires during an active exam session, a warning counter is incremented and compared against the administrator-configured threshold. If the threshold is not yet reached, the student is shown a warning and the exam continues; if it is reached, the exam is automatically submitted and the session is closed.

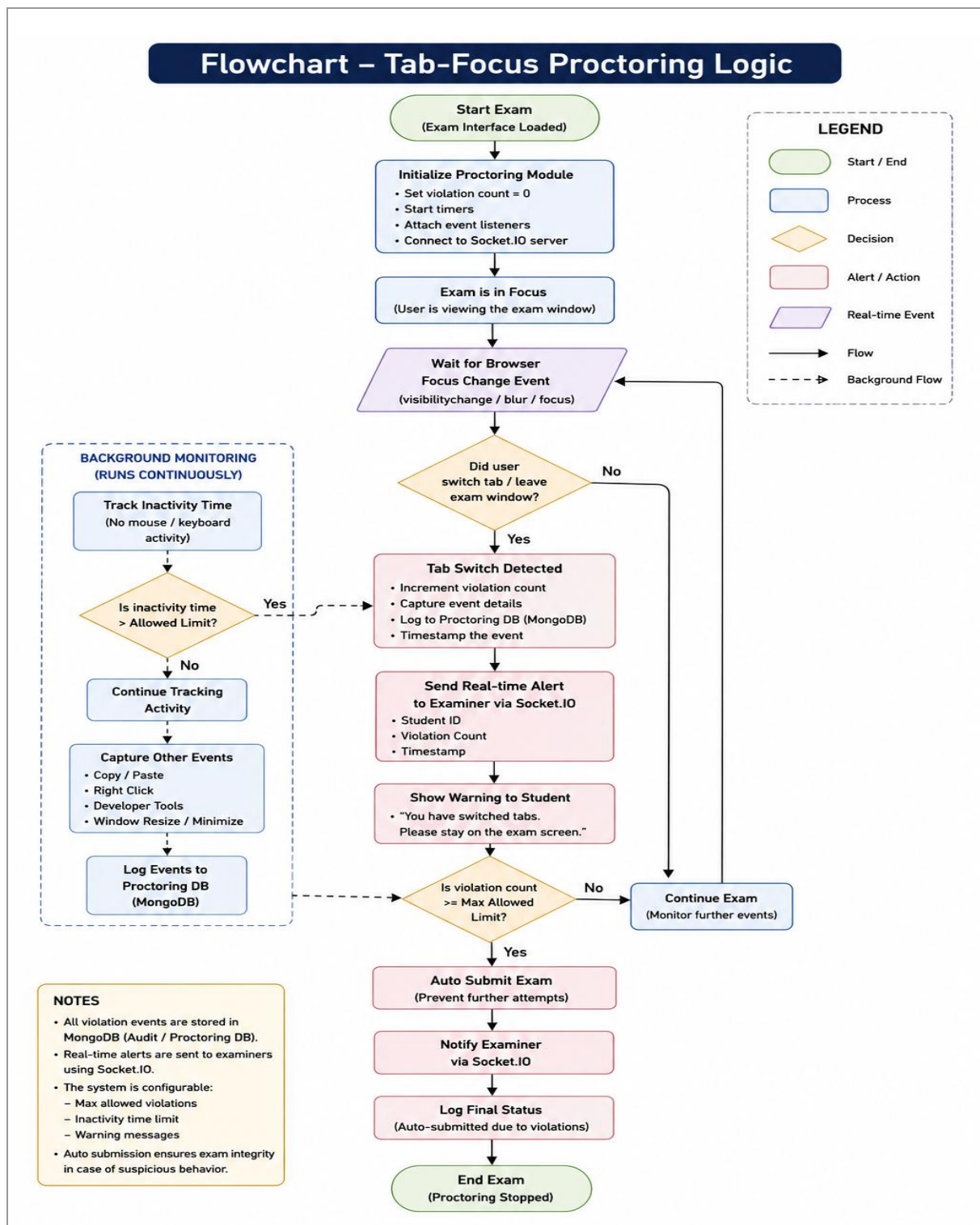


Figure 6.5: Flowchart – Tab-Focus Proctoring Logic

6.5 Pseudo Code

The following pseudo code summarises the client-side proctoring handler described above.

FUNCTION onVisibilityOrBlurEvent():

```
IF examSession.isActive THEN

    warningCount = warningCount + 1

    LOG proctoringEvent(type, timestamp) TO auditBuffer

    IF warningCount >= maxAllowedSwitches THEN

        CALL submitExam(auditBuffer, autoSubmitted = true)

        CLOSE examSession

    ELSE

        DISPLAY warning(warningCount, maxAllowedSwitches)

    END IF

END IF

END FUNCTION
```

7. TESTING

7.1 Functional Testing

Functional testing verified that each feature described in the requirements of Chapter 5 behaved as specified. This included confirming that registration correctly associates a student with a chosen examiner, that JWT-protected routes reject requests carrying an invalid or expired token with an HTTP 401 response, that the tab-focus counter increments correctly on both the `visibilitychange` and `blur` events, and that auto-submission triggers precisely at the configured threshold rather than one event before or after it.

7.2 Structural Testing

Structural (white-box) testing focused on the internal logic of the authentication middleware, the proctoring threshold comparison, and the controller layer responsible for persisting submissions across the two database stores. Particular attention was paid to boundary conditions in the warning-counter logic and to ensuring that a failure writing to the MongoDB audit store did not prevent the primary SQLite submission from being saved, since the two writes serve different purposes and a failure in one should not silently corrupt the other.

7.3 Levels of Testing

Testing was carried out at three levels, summarised in Table 7.2: unit-level tests exercised individual functions such as JWT signing and verification in isolation; integration-level tests verified that the Express controllers, SQLite queries, and MongoDB writes worked correctly together end to end; and system-level tests exercised complete user journeys — registration through exam submission and grading — against a running Docker Compose stack to approximate the production environment as closely as possible.

Level	Focus	Example
Unit	Individual functions in isolation	JWT signing/verification, warning-counter increment logic
Integration	Interaction between backend controllers and both databases	Exam submission writing to SQLite and MongoDB together
System	End-to-end user journeys against a running deployment	Registration → exam attempt → auto-submit → grading → result view

7.4 Testing the Project

A representative sample of the functional test cases executed against the running application is listed in Table 7.1. Each case records the precondition, the action performed, and the expected outcome observed during manual verification.

Test Case	Steps	Expected Result
Invalid login	Submit login form with an incorrect password	HTTP 401 returned; error message displayed; no token issued
Tab-switch warning	Switch to a second browser tab during an active exam once	Warning counter increments to 1; on-screen warning shown; exam remains active
Auto-submit on threshold breach	Switch tabs until the configured maximum is reached	Exam is submitted automatically; session is closed; violation logged with timestamp
Role-guarded route	Attempt to access the Admin panel while logged in as a Student	Redirected to /unauthorized; no admin data returned by the API
Grade publication	Examiner grades a submission and clicks Publish	Result becomes visible on the corresponding student's dashboard

8. IMPLEMENTATION

8.1 Implementation of the Project

Implementation followed the phased plan set out in Chapter 4, beginning with the authentication and role-based routing layer, since almost every other feature in the system depends on a working identity and permission model. The frontend was implemented in React 19 using functional components and context providers for global authentication state, bootstrapped and bundled through Vite. Styling relies on vanilla CSS with a set of shared design variables to keep the glassmorphic visual language consistent across the Student, Examiner, and Admin portals, while Framer Motion drives page-transition and micro-interaction animations. Lucide React supplies the icon set, and network communication uses the native Fetch API wrapped in a small custom service module that attaches the bearer token to every outgoing request, approximating the interceptor pattern more commonly associated with Axios.

On the backend, Express routes were organised by resource — authentication, exams, submissions, and administration — with a shared authMiddleware verifying JWTs on every protected route. SQLite was chosen for the primary application store because of its file-based simplicity during development and its adequacy for the transactional read/write volume expected of a single-institution deployment; MongoDB, accessed through Mongoose, handles the audit log, storing each proctoring event as a flexible document that can carry different fields depending on the event type without requiring a schema migration. Socket.io was layered on top of the same Express HTTP server to push tab-switch warnings and snapshot events to any examiner dashboard actively subscribed to that exam session, avoiding the latency and server load associated with polling.

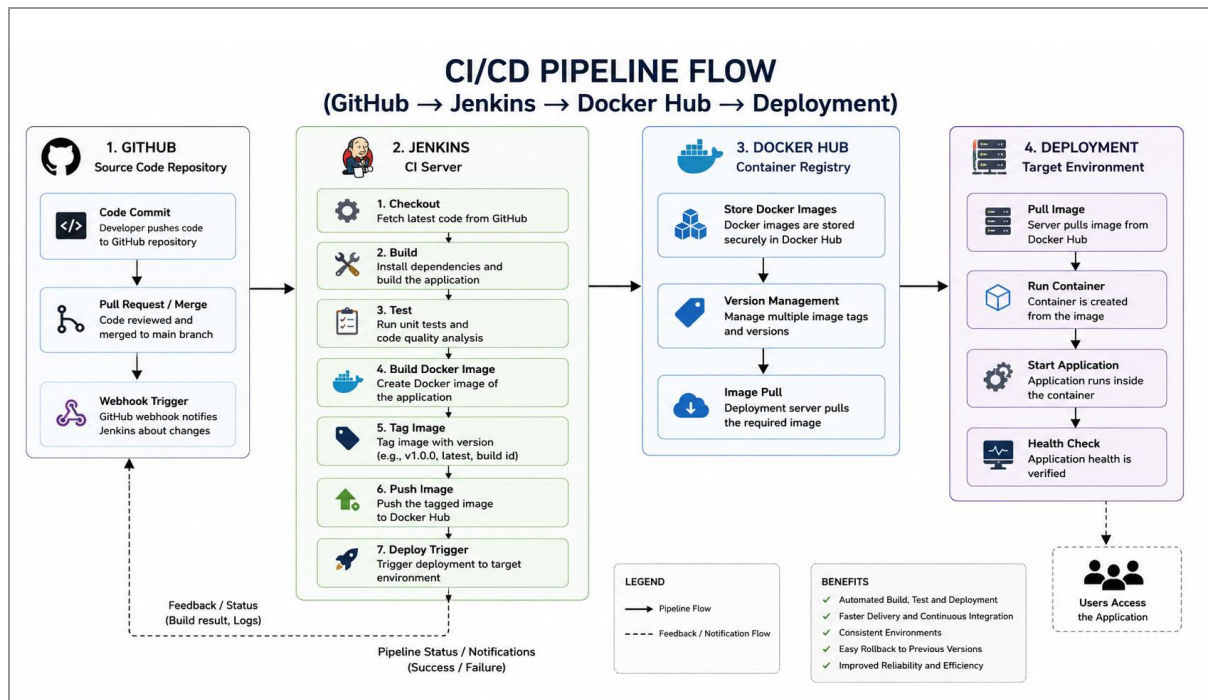


Figure 8.1: CI/CD Pipeline Flow (GitHub → Jenkins → Docker Hub → Deployment)

Containerisation used a multi-stage Dockerfile for the frontend, compiling the Vite build in a Node.js build stage before copying the resulting static assets into a lightweight Nginx image responsible for serving them and reverse-proxying API calls to the backend container. The backend Dockerfile installs production dependencies and launches the Express server directly. Docker Compose ties both services together on an isolated bridge network, mapping host port 3000 to the frontend container and host port 5002 to the backend, with a named volume preserving the SQLite database file across container restarts. A declarative Jenkinsfile, triggered by a GitHub webhook relayed through an Ngrok tunnel to a locally hosted Jenkins instance, automates environment verification, dependency installation, the Vite build, Docker image builds, and pushes to Docker Hub, before pulling the freshly published images and redeploying the containers.

8.2 Conversion Plan

Because ExamProctor was developed as a new system rather than a replacement for an existing digital platform, no data migration from a prior application was required. The conversion effort instead centred on onboarding: seeding the database with initial administrator credentials, importing an examiner's existing student roster through the registration workflow, and validating that the Docker Compose stack could be brought up cleanly on a fresh host using the provided deploy.sh script, without any manual post-installation configuration beyond populating environment variables.

8.3 Post-implementation and Software Maintenance

Once deployed, ongoing maintenance is handled largely through the existing CI/CD pipeline: a change pushed to the main branch triggers the Jenkins job automatically, rebuilding and redeploying the affected containers without manual intervention. Routine maintenance tasks available to the Admin/Super Admin role include reviewing system logs, adjusting the proctoring sensitivity and maximum tab-switch threshold, and performing a controlled database reset when needed for a new academic term. Because the SQLite database file is preserved on a named Docker volume independent of the container lifecycle, redeploying updated application images does not risk data loss.

9. PROJECT LEGACY

9.1 Current Status of the Project

At the time of writing, ExamProctor implements the complete feature set defined in the requirements of Chapter 5: role-based authentication for Student, Examiner, and Admin/Super Admin; exam creation and configuration; tab-focus proctoring with configurable thresholds and automatic submission; real-time violation notification through Socket.io; grading and result publication; and a fully containerised deployment automated through Jenkins. The system has been tested end to end against a Docker Compose deployment mirroring the intended production configuration.

9.2 Remaining Areas of Concern

A small number of areas were identified as needing further attention beyond the scope of this project term. The webcam snapshot feature currently transmits base64-encoded images directly to the backend rather than to dedicated object storage, which is adequate for a small deployment but would not scale efficiently to a large concurrent candidate pool; this is addressed as a proposed enhancement in Chapter 12 of the original architecture documentation, migrating snapshot storage to a service such as AWS S3. Additionally, the current single-host Docker Compose deployment model does not provide horizontal scaling; a Kubernetes-based deployment was considered but placed outside this project's scope given the additional infrastructure complexity it would introduce.

9.3 Technical and Managerial Lessons Learnt

From a technical standpoint, the project reinforced the importance of separating concerns between transactional and audit data early in the design process, rather than retrofitting an audit trail onto a schema that was not built to accommodate it. It also highlighted the value of automated deployment pipelines even for a project of modest scale: once the Jenkins pipeline was in place, the overhead of testing incremental changes against a realistic deployment dropped substantially compared to manually rebuilding and restarting containers after every change. From a managerial perspective, bounding the scope explicitly in Chapter 2 — deferring video proctoring and mobile applications rather than attempting to include them — proved essential to completing a working, tested system within the available project timeline. Attempting to include every feature considered during initial planning would have left several components partially implemented rather than a smaller set of features working reliably end to end.

10. BIBLIOGRAPHY

- [1] Meta Platforms, Inc., “React – The Library for Web and Native User Interfaces,” React Documentation. [Online]. Available: <https://react.dev>
- [2] Vite Contributors, “Vite – Next Generation Frontend Tooling,” Vite Documentation. [Online]. Available: <https://vitejs.dev>
- [3] OpenJS Foundation, “Node.js Documentation,” Node.js. [Online]. Available: <https://nodejs.org/en/docs>
- [4] Express.js Contributors, “Express – Node.js Web Application Framework,” Express Documentation. [Online]. Available: <https://expressjs.com>
- [5] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [6] D. Hardt, “The OAuth 2.0 Authorization Framework,” RFC 6749, Internet Engineering Task Force, Oct. 2012. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6749>
- [7] MongoDB, Inc., “MongoDB Manual,” MongoDB Documentation. [Online]. Available: <https://www.mongodb.com/docs/manual>
- [8] SQLite Consortium, “SQLite Documentation,” SQLite. [Online]. Available: <https://www.sqlite.org/docs.html>
- [9] Socket.IO, “Socket.IO Documentation,” Socket.IO. [Online]. Available: <https://socket.io/docs>
- [10] Docker, Inc., “Docker Documentation,” Docker Docs. [Online]. Available: <https://docs.docker.com>
- [11] Jenkins Contributors, “Jenkins User Documentation,” Jenkins. [Online]. Available: <https://www.jenkins.io/doc>
- [12] Nginx, Inc., “Nginx Documentation,” Nginx Docs. [Online]. Available: <https://nginx.org/en/docs>

[13] OWASP Foundation, “OWASP Top Ten,” OWASP. [Online]. Available: <https://owasp.org/www-project-top-ten>

[14] Mozilla Developer Network, “Page Visibility API,” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Page_Visibility_API